

CS 64 HANDOUT #5

MIPS Assembly: Programming with Functions and the MIPS Calling Convention

Prof. Ziad Matni

Read this handout **before** coming to class. In lecture, I will expand a lot on the concepts in this document, and give lots of examples. So, these handouts are a good explanation of what I'll be talking about in class, but they are not good substitutes for missing the lecture! ☺

FYI: This particular handout is a long and sometimes complex read.

In this reading, we will discuss the following topics:

- The MIPS Calling Convention – Part 1: Basics
- Basic Function Calls
- The Call Stack
- The MIPS Calling Convention – Part 2: Functions that Call Functions
- The MIPS Calling Convention – Part 3: Register Preservation

1. The MIPS Calling Convention – Part 1: Basics

A **convention** is simply an agreed-upon way of doing things – contrast this with a **rule**, which is a *codified guideline* in programming that comes with explicit consequences for violations (i.e., if you don't follow a rule, your program will simply not work).

So, you may be curious to know, as far as conventions go, *who is it exactly* that “agrees upon” what needs to happen? The answer is (at least in programming): the professional software engineering community and the international professional organizations that sponsor all manner of conventions in the software engineering world (e.g., ACM, IEEE, etc.) A common follow-up question may be: *why??* The answer here is: *intercompatibility* of software products and of CPU usage. If everyone agrees to certain conventions, then it makes programming work amongst a team of programmers much easier, including the development part of a software project *and* the testing part of that project.

The goal of any calling convention is to *have a way of implementing functions in assembly*. This is actually more challenging than it may initially sound (!!), as there is nothing inherently built into the CPU that allows for function calls to be performed, at least in a way that is as straightforward as, say, adding numbers together. For example, with MIPS, while we have a way to start executing some other piece of code (as we need for a function), we do not have an inherent way of jumping back to some other piece of code when execution has completed elsewhere. As another example, while we have a way to save variables somewhere with MIPS (i.e., with registers), we don't have an inherent way to save an arbitrary number of variables. Without this capability, we can have neither arbitrarily nested function calls, nor can we have recursion.

While MIPS *does not* have built-in capabilities which handle functions, it does provide enough functionality to implement functions ourselves. For example, MIPS allows for memory access, which, when used with care, can be used to implement a *call stack*. Additionally, with some care, we can explicitly have code jump back to a caller, effectively implementing a return in the process. As such, we are not stuck in the water when it comes to functions: it's just that we need to put *more* effort into their implementation, as functions exist at a *higher level of abstraction* than what is provided by the MIPS hardware. **This also means that there are many possible ways to implement functions, each with its own set of advantages and disadvantages.** For correctness, it is important that all functions are implemented in the *same* way, as function calls require specific interactions between different pieces of code.

Registers play a special role in the calling convention; this is ultimately why the different registers have the names they do (e.g., **\$t0** versus **\$s0**: **t*** taken to mean “temporary register”, while **s*** here means “saved register”).

The first 2 rules we'll learn about the MIPS Calling Convention are:

- a) All function argument values must be placed in registers **\$a0** thru **\$a3** before the function is called. This assumes that the function only has 4 or fewer arguments. The function *caller* is responsible to set this up before the **jal** instruction is given.
- b) Function return values are expected to be in registers **\$v0** and **\$v1** before the function returns to the *caller*. This assumes that the function will return 2 or fewer values. The function itself is responsible to set this up before the **jr** instruction is given.

2. Basic Function Calls

We all know what *functions* are in computer programming! So, let's recall what are the capabilities we need to have in order to use functions in a program:

- a) We need the ability to **call** a function and **execute its code** that lies elsewhere in the program. You already know how branches and jumps aid in this regard in MIPS assembly.
- b) We need the ability to **pass arguments** into a function and **get return values** from function. This can be done in different ways in MIPS assembly, *however* there's a *preferred* way to do it. This preferred way is called the *MIPS Calling Convention*.

We've learned that, in MIPS assembly programming, the **j** instruction jumps unconditionally to another part of the program (e.g., **j label_x**). But when programming a function, you need a return address – i.e., a place in the program to come back to after the function is finished executing. Therefore, you also need a mechanism to aid *storing* that return address for you when you first call the function.

In MIPS, this is accomplished with the use of the “jump-and-link” and “jump-register” instructions (**jal** and **jr**) and the return address register (**\$ra**). The **jal** instruction simultaneously does 2 things: it has the CPU jump to another part of the program (i.e., it jumps to some address by use of a label), and it stores the location of the *next instruction* after the

function call in register **\$ra**. The **jr** instruction is typically placed at the *end* of the function routine so that the CPU can then return to the (now saved) next instruction that's after the function call. The address of the return address is stored in register **\$ra**.

Consider this example C++ function:

```
int DosArguments(int x, int y) {  
    return x + y;  
}
```

Consider the following function call to `DosArguments`:

```
DosArguments(3,4)
```

The above function call can be implemented like so:

```
li $a0, 3  
li $a1, 4  
jal DosArguments
```

That is, arguments are specified in registers **\$a0** through **\$a3**. The first argument is passed in **\$a0**, the second argument in **\$a1**, and so on. The way values are passed is to actually set the corresponding register to the appropriate value. In this case, because the first argument was a 3, we set register **\$a0** to the value 3 immediately before the call to `DosArguments`. As for the definition of `DosArguments`, it will then look at **\$a0** and **\$a1** to get its parameter values.

In regards to the returned value (**x + y**, which in this case is the integer 7), recall that the convention says the registers to be used are **\$v0** and **\$v1**. The first value returned is placed in register **\$v0** (if applicable), and the second value returned is placed in register **\$v1** (if applicable). With this in mind, we can implement `DosArguments` in MIPS assembly, like so:

```
DosArguments:  
    add $v0, $a0, $a1  
    jr $ra
```

Keep in mind that we only have registers **\$a0** through **\$a3** available to us, meaning functions can take at most 4 arguments (which would be passed in all four registers). You should never need more than 4 arguments in this course. There is a way to pass more via other mechanisms, though we won't be discussing them here. Likewise, in this course, you should never need more than 2 returned values.

Consider this full program with a simple function call example:

```
.text
DosArguments:
    # this function expects the arguments to be in registers $a0-$a3
    addi $v0, $a0, $a1

    # return to whoever made the call
    # the return value is expected to be in register $v0
    # the return address is expected to be in register $ra
    jr $ra

main: # set up the parameters to call the function
    li $a0, 6
    li $a1, -2
    # then call the test function
    jal test

    # do something with the returned value, e.g. print it
    move $a0, $v0
    li $v0, 1
    syscall

exit: # exit program
    li $v0, 10
    syscall
```

Note that the **main:** label is not the first one in the program text – and that’s ok. This is akin to having functions defined before the main function of a program in C++, Python, etc... When you run this program in the MIPS emulator, the program execution will start at the **main:** label.

So far, the functions we have implemented have been relatively simple. Let's make things a bit more complex, by introducing the need for *temporary variables*, like so:

```
int needsTemps(int x, int y) {
    int add = x + y;
    int sub = x - y;
    int mult = add * sub;
    return mult;
}
```

The function **needsTemps** above uses the temporary variables **add**, **sub**, and **mult**. Previously in this sort of situation, you simply would use some arbitrary registers here. However, for the purposes of following the MIPS calling convention, for reasons we'll get into shortly, we are a bit restricted with what we can use. In this case, we can use registers **\$t0** through **\$t9**. For translating the above code, we'll use the following temporary variable to register mapping.

Other than the fact that we restrict ourselves to registers **\$t0** - **\$t9**, this mapping is arbitrary; we can use any of the variables in the range **\$t0** - **\$t9**.

For simplicity's sake, let's say we will map the variables **add**, **sub**, and **mult** to **\$t0**, **\$t1**, and **\$t2**.

So, we can translate **needsTemps** into the following MIPS assembly code:

```
needsTemps:
    add $t0, $a0, $a1
    sub $t1, $a0, $a1
    mult $t0, $t1
    mflo $t2
    move $v0, $t2
    jr $ra
```

The above MIPS code is a straightforward translation of the C++ code, which follows the C++ code exactly. Of course, we can slightly simplify this further by eliminating the need for the temporary register **\$t2** and thereby making the code a little more efficient by eliminating the pseudoinstruction **move**, like so:

```
needsTemps:
    add $t0, $a0, $a1
    sub $t1, $a0, $a1
    mult $t0, $t1
    mflo $v0
    jr $ra
```

We are going to learn more about the MIPS Calling Convention, especially as it pertains to function parameters and the registers used for them when we have cases of *nested functions*, that is, functions that call other functions (this includes *recursive functions*, of course).

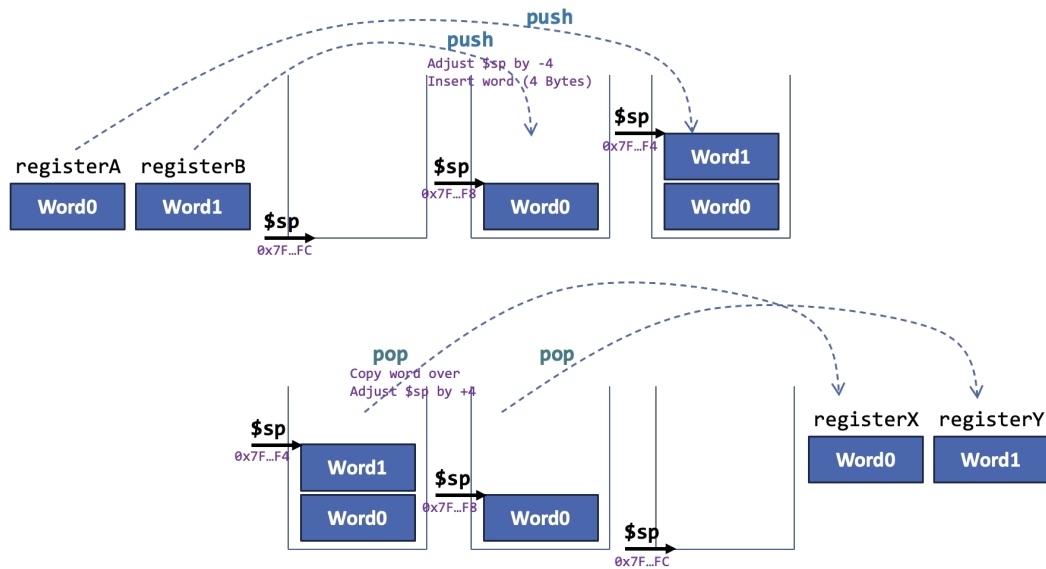
However, let's take a little detour first and discuss the call stack in CPU memory.

3. The Call Stack

When a program starts executing, a certain contiguous section of memory is set aside for the program called the **stack**. As you may have learned in other CS courses, control of the stack is typically operated automatically in high-level programming languages (in contrast with the explicit use of dynamic data memory – also known as the **heap** – by the programmer). You may recall that data is added and retrieved to/from stack memory in **LIFO** fashion (Last-In-First-Out) via operations known as “**push**” and “**pop**”, respectively.

The stack memory in a CPU is allocated a certain space and the location of the “top of the stack” is usually pointed to by a “stack pointer”. There are 2 things we need to know about how MIPS CPU memory is constructed to make use of the stack part of memory:

- In MIPS, there is a dedicated register for stack pointer (**\$sp**). The value in this register should always point to the top of the stack. Upon startup, the default initial value in **\$sp** is **0x7FFFFFFC**.
- In MIPS, the stack part of memory is meant to be used starting from high-addresses and going to low-addresses (i.e., you build it *upside down!*). The “bottom” of the stack memory is always the default initial value of **\$sp** (i.e., **0x7FFFFFFC**).



When operating a **push operation**, the CPU is thus instructed to do 2 main things: (1) adjust the stack pointer to where the new top of the stack *is going to be* (i.e., make the stack pointer point to what the new memory address of this is), and (2) insert the data from one or more registers into stack memory. *Remember that: (a) MIPS addressing is done in BYTES and (b) Big Endian rules apply here!* As an example, when wanting to push 1 word residing in register **\$s0** into stack memory, the assembly code should be:

Push 1 word into the stack:

```
addiu $sp, $sp, -4    # Adjust the stack pointer by 4 bytes (1 word)
sw $s0, 0($sp)       # Store the word in $s0 to the stack memory
```

Another example: to push 3 MIPS words residing in registers **\$s0**, **\$s1** and **\$s2** into stack memory, the assembly code is as follows:

Push 3 words into the stack:

```
addiu $sp, $sp, -12   # Adjust the stack pointer by 12 bytes
sw $s0, 8($sp)        # Store the word in $s0 to the stack memory
sw $s1, 4($sp)        # Store the word in $s1 to the stack memory
sw $s2, 0($sp)        # Store the word in $s2 to the stack memory
```

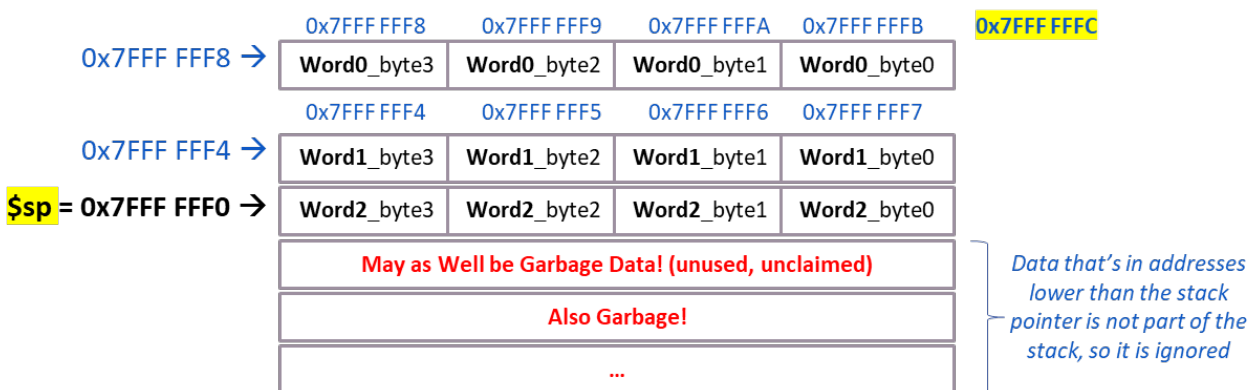
Likewise, when operating a **pop operation**, the CPU has to reverse these tasks: (1) removes (copies over) the data from stack memory into one or more registers, and (2) adjust the stack pointer to where the new top of the stack will be. For example, to pop 3 MIPS words from stack memory into registers **\$s0**, **\$s1** and **\$s2**, the assembly code is as follows:

Pop 3 words into the stack:

```
lw $s0, 8($sp)      # Load the word in $s0 from the stack memory
lw $s1, 4($sp)      # Load the word in $s1 from the stack memory
lw $s2, 0($sp)      # Load the word in $s2 from the stack memory
addiu $sp, $sp, 12   # Adjust the stack pointer by 12 bytes
```

You'll note from the example codes given above that adjusting the stack pointer in *push* operations involves *subtraction* of some value from a memory address, and conversely in *pop* operations, it involves *addition*. This is because, as mentioned earlier, the stack gets built "upside down" – that is, the bottom of the stack is the highest address in stack memory (the default **0x7FFFFFFC**). The same principles of stack data structures apply here – it's just that the memory addresses progress from high to low.

Here is a diagram that illustrates the stack after a 3-word push operation:



4. The MIPS Calling Convention – Part 2: Functions that Call Functions

In this section, we'll introduce the portion of the MIPS calling convention which is arguably the most crucial, but also sometimes the more difficult one to understand: *functions which call functions*. To illustrate, consider the following C++ code:

```
void bar() { return; }
void foo() {
    bar();
    return;
}
int main() {
    foo();
    return 0;
}
```

Using the components you've seen so far, you may be tempted to implement first and second using the following MIPS assembly code:

```
bar:
    jr $ra
foo:
    jal bar
    jr $ra
main:
    jal foo
    # exit program
    li $v0, 10
    syscall
```

The above code, however, does not work correctly!

To understand why, recall that **\$ra** is intended to hold the address of the instruction to return back to. When **main** calls **foo** with the “**jal foo**” instruction, the address of the instruction after the call (specifically the address of the **li \$v0, 10** instruction) is put into register **\$ra**. This becomes problematic when **foo** then calls **bar**, because this then puts the next instruction’s address into **\$ra** (specifically the **jr \$ra** instruction in **foo**). This **overwrites** the value in **\$ra**!! When **bar** returns (back to what called it), this is not a problem, **but** now **foo** no longer has the appropriate address to return to because it was overwritten when **foo** called **bar**. Now, when **foo** executes **jr \$ra**, it's going to start executing instructions where **\$ra** currently points, which just so happens to be the **jr \$ra** instruction in **foo**. As such, this code will now forever jump to the same instruction over and over again - the **jr \$ra** instruction in second and the program gets stuck in that repeated action.

A (Broken) Workaround

A potential workaround is to put the address to return to in another register. To illustrate this idea, consider the code below, which shows changes from the previous code in bold.

Note that this is just to illustrate the idea; you should never implement your code this way, as it violates the calling convention.

```
bar:
    jr $ra
foo:
    move $t0, $ra
    jal bar
    jr $t0
main:
    jal foo
    # exit program
    li $v0, 10
    syscall
```

The key change with the code above is that **foo** copies the value of **\$ra** into **\$t0** before the call to **bar**, and then it returns to **main** by jumping to the address specified in **\$t0**. This ends up working around the original problem, because now **\$ra** isn't being overwritten.

The problem with this workaround, however, is that *it does not scale up to larger programs*. Eventually, we'd completely run out of registers. Moreover, this strategy doesn't allow for *recursive functions*, which need to jump to the same function label over and over again.

The Correct Solution: Use the MIPS Calling Convention

The solution to the above problem is to use the call stack in CPU memory. The call stack is a type of data structure in memory which stores the return addresses of the various functions which have been called in the past as well as any other values that needed to be saved on purpose from one function call to the next.

Recall that when a function is called, the address that the function needs to return to, is stored in the **\$ra** register. As we're about to find out, the MIPS calling convention (i.e., the "correct solution") says *the program must store the return address in the call stack*. More specifically, the convention states that if a function (the "caller") calls another function (the "callee"), the "callee" must save (i.e., *push*) the value of register **\$ra** onto the call stack. When a function returns, the address is taken from (or more specifically, *popped off of*) the call stack. Let's illustrate that with the prior example and using *fake* code – *just for illustration purposes* (note that **push** and **pop** are not real MIPS instructions):

```
bar:
    jr $ra
foo:
    # save the $ra value (address of "li $v0,10") in the stack
    push $ra
    jal bar    # remember that this instruction overwrites $ra
    # to prepare the return to the caller function,
    # retrieve the $ra value from the stack
    pop $ra
    jr $ra
main:
    jal foo
    # exit program
    li $v0, 10
    syscall
```

In the above fake (pseudo) code, the intended semantics of **push \$ra** is to push the current value of **\$ra** onto the call stack. The intended semantics of **pop \$ra** is to pop the topmost value of the call stack into register **\$ra**.

As we mentioned in a previous section of this reading, **push \$ra** and **pop \$ra** are stand-ins for actual MIPS assembly instructions that transform the code (correctly now) into the following:

```
bar:
    jr $ra
foo:
    # push $ra into stack:
    addiu $sp, $sp, -4
    sw $ra, 0($sp)
    jal bar    # remember that this instruction overwrites $ra

    # pop value from stack into $ra:
    lw $ra, 0($sp)
    addiu $sp, $sp, 4
    jr $ra
main:    # The rest of the code is not shown...(it's unchanged)
```

So, let's add a *new rule* to the 2 previous MIPS Calling Convention rules:

- a) All function argument values must be placed in registers **\$a0** thru **\$a3** before the function is called. This assumes that the function only has 4 or fewer arguments. The function *caller* is responsible to set this up before the **jal** instruction is given.
- b) Function return values are expected to be in registers **\$v0** and **\$v1** before the function returns to the *caller*. This assumes that the function will return 2 or fewer values. The function itself is responsible to set this up before the **jr** instruction is given.
- c) **NEW RULE!!!**
If a function ("caller") is calling another function ("callee"), it has to first save the current value of the return address (located in register **\$ra**) in the stack. The "caller" function has the responsibility of saving **\$ra** in the stack with a **push** operation at the *beginning* of that function. The "caller" function also has to retrieve the **\$ra** value using a **pop** operation on the stack at the *end* of that function, just before it issues a **jr \$ra** instruction.

5. The MIPS Calling Convention – Part 3: Register Preservation

The previous example above shows how to manipulate the call stack in order to have arbitrarily deep chains of function calls working properly. However, it does not address the needs of functions which need *additional registers* for processing. For example, consider the following C++ code:

```
int subTwo(int a, int b) {
    int sub = a - b;
    return sub;
}

int doSomething(int x, int y) {
    int a = subTwo(x, y);
    int b = subTwo(y, x);
    return a + b;
}

# Some other function calls doSomething()...
```

We've previously said that, if additional registers were needed, then we could use the registers **\$t0** - **\$t9** (remember the "t" here stands for "temporary"). However, this is problematic with the above code. For example, within **doSomething()**, we may want to map variables **a** and **b** to registers **\$t0** and **\$t1**, respectively. This mapping would mean that **subTwo()** could not map variable **sub** to either register **\$t0** or **\$t1**, as doing so would interfere with the values that **doSomething()** puts into these 2 registers. While we can get around this problem in *this* case by being sure to map variable **sub** to a *different* register (e.g., **\$t2**), this is another hacky workaround (which, as you know, *you should never do* in your code, as it violates the MIPS calling convention). With enough function call nesting, we'll once again eventually run out of registers! So, what do we do??

Once again, the call stack comes to the rescue! In addition to pushing the values of **\$ra** onto the stack when we want to preserve them across calls, we can *also* push other values, as with variables **a** and **b** above. This is shown in the MIPS code below, which implements **subTwo()** and **doSomething()** legally according to the MIPS calling convention (however, it is *not* an optimized code at all – more on that soon...)

Implementing a register preservation scheme, per the MIPS calling convention:**subTwo:**

```
# Map variable sub to $v0 (since it's our returned value)
sub $v0, $a0, $a1
jr $ra
```

doSomething:

```
# Allocate space for values of $ra, var. x ($a0), and var. y ($a1),
# which all must be there after the function call to subTwo.
# Save these values onto the stack (push).
```

```
addiu $sp, $sp, -12
sw $ra, 8($sp)
sw $a0, 4($sp)
sw $a1, 0($sp)
```

```
# x and y are already in the right places for the first call to
# subTwo, so we can call it directly:
```

```
jal subTwo
```

```
# After we come back from this first call, we need the (unchanged)
# values of x and y for our second call to subTwo. We do NOT know
# for sure if subTwo over-wrote these var. values (no guarantee).
# So, we re-load x and y values (to be sure) from the stack.
# Note 1: we don't have to pop them, just copy them over
#           but leave them in the stack
# Note 2: we intentionally flip where we place these values,
# as the call is to subTwo(y, x)
```

```
lw $a0, 0($sp)
lw $a1, 4($sp)
```

```
# Store var. a on the stack, which is held in $v0
# (which the first subTwo call returned) b/c we need to
# preserve that value for after the second call.
```

```
addiu $sp, $sp, -4
sw $v0, 0($sp)
```

```
# NOW make the second call to subTwo
```

```
jal subTwo
```

```
# the result of subTwo(y, x) (var. b) is in $v0.
```

```

# Add this to the previously stored var. a
#      (which we'll need to grab from the stack)
# Note again that we don't have to pop it, just copy it over
#      but leave it in the stack

lw $t0, 0($sp)
add $v0, $v0, $t0      # prep return value directly

# We now load in the value of $ra from the stack.
# Note that the offset is different from before,
# since we allocated additional values on the stack

lw $ra, 12($sp)

# deallocate space on the stack, and return

addiu $sp, $sp, 16
jal $ra

```

The above code is valid, and it obeys the MIPS calling convention (with a **big caveat**...). In general, you should remember the following: Values in unreserved registers (every register introduced so far except for **\$sp**), are **not** preserved across function calls. In other words, you must **always assume** that non-preserved registers can change values across function calls, even if you know for a fact that they don't...!

This convention means that functions operate independently of each other when it comes to how registers are treated. Violations of the above rule in correct code necessarily mean that one function relies on another function not modifying a register, which leads to hacky code (functions should be able to change interdependently of each other).

However (and here's the **big caveat**), if you thought this code was a little convoluted and complex, *you're not mistaken!!* So, let's re-analyze the code and think of better (more efficient) ways to accomplish the same thing.

Nearly all of the instructions in **doSomething** are devoted to manipulating the call stack, which is *not ideal*. Additionally, reasoning about *which values* are on the call stack at *which times* is not trivial. For example, while the value of **\$ra** is stored onto the stack initially at offset **8** of **\$sp**, we later load the value of **\$ra** from offset **12** of **\$sp**, as another 4-byte stack allocation had taken place since that point. *It would be much nicer if we could reference individual variables in the code via registers*, which are more permanent than offsets from the stack pointer (**\$sp**).

It is here where registers **\$s0 - \$s7** come in. These registers are meant to be **preserved**, meaning that **their values persist across calls**. This is in stark contrast to registers **\$t0 - \$t9**, whose values *do not persist* across calls.

The downside of registers **\$s0** - **\$s7** is that if you want to use one of them, it is up to you to save the register's current value and *then* later restore it. This way, the “caller” code never “sees” the value of an **\$s*** register change (let this statement sink in: it’s important)! In contrast, registers **\$t0** - **\$t9** can be written to directly at all times with no ill consequence to the MIPS calling convention, since we are told to assume that the values in these registers will *not persist* across function calls.

So, let’s add *another* (and *final*) rule to the 3 previous MIPS Calling Convention rules and I’ll highlight the main aspect of what each rule is about:

- a) All function argument values must be placed in registers **\$a0** thru **\$a3** before the function is called. This assumes that the function only has 4 or fewer arguments. The function *caller* is responsible to set this up before the **jal** instruction is given.
- b) Function return values are expected to be in registers **\$v0** and **\$v1** before the function returns to the *caller*. This assumes that the function will return 2 or fewer values. The function itself is responsible to set this up before the **jr** instruction is given.
- c) If a function (“caller”) is calling another function (“callee”), it has to first save the current value of the return address (located in register **\$ra**) in the stack. The “caller” function has the responsibility of saving **\$ra** in the stack with a **push** operation at the *beginning* of that function. The “caller” function also has to retrieve the **\$ra** value using a **pop** operation on the stack at the *end* of that function, just before it issues a **jr \$ra** instruction.
- d) **NEW RULE!!!!**
The **only** register values that are assumed to **persist** across a function call are the **\$s0** - **\$s7** registers. The only way to preserve them is by using the call stack. ALL OTHER registers are assumed to NOT persist across function calls – this includes the **\$a**-registers, **\$t**-registers, and **\$v**-registers. We are not counting the **\$ra** and the **\$sp** registers, which are purposely changed when pushing or popping into/from the stack.

To illustrate, let’s re-work the previous example with registers **\$s0** - **\$s7** and you can see how it simplifies a lot of the previous assembly code.

Implementing a **BETTER** register preservation scheme, per the MIPS calling convention:**subTwo:**

```

# Map variable sub to $v0 (since it's our returned value)
sub $v0, $a0, $a1
jr $ra

```

doSomething:

```

# Map vars. x, y, and a, into $s0, $s1, and $s2, respectively.
# Optimization: we don't need to preserve the value in var. b
# because it does not need to "live" beyond a call.
# So, we allocate space for 4 registers (including $ra) and push
# their values into the stack to preserve their old values:

```

```

addiu $sp, $sp, -16
sw $s0, 0($sp)
sw $s1, 4($sp)
sw $s2, 8($sp)
sw $ra, 12($sp)

```

```

# Initialize $s0 and $s1 with values of x and y, respectively.
# We are overwriting whatever values may have been on $s0 & $s1
# and that's ok b/c we saved their old values in the stack!

```

```

move $s0, $a0
move $s1, $a1

```

```

# Call subTwo(x,y)
jal subTwo

```

```

# The returned value is in var. a ($v0), which we need to preserve
# across the second subTwo call, but since v-regs are not preserved,
# we need to copy this value over to an s-reg ($a2)

```

```

move $s2, $v0

```

```

# Prepare for second call: subTwo(y, x), then compute final result:

```

```

move $a0, $s1
move $a1, $s0
jal subTwo
add $v0, $v0, $s2

```

```

# Restore registers, deallocate stack memory, and return

```

```

lw $ra, 12($sp)
lw $s2, 8($sp)
lw $s1, 4($sp)
lw $s0, 0($sp)
addiu $sp, $sp, 16
jr $ra

```

Note how this newer code is a lot easier to follow and looks “cleaner”!

The above code now utilizes registers **\$s0** - **\$s7** in order to have values effectively survive past the calls to **subTwo**.

A Strategy for Mapping Variables

Let us now outline a strategy for mapping variables to different registers. If a variable's value does not need to “live” (i.e., be available) beyond a function call, then it can be safely mapped to “temporary” registers **\$t0** - **\$t9**. While there is no restriction on which registers you map to here, it is *generally preferable* to map variables to registers *in the order that they appear* in the program. For example, with the C++ program code...

```
void foo() {  
    int x = 2 + 2;  
    int y = 7 * 8;  
}
```

...perhaps the most straightforward mapping is to map variables **x** to **\$t0** and **y** to **\$t1**.

If a variable's value does need to “live” beyond a call, then generally it should be placed in one of the **\$s*** registers. Again, it is generally better to map variables to registers in the order in which they appear in the original program.

An Important Note re: our Course:

Note that if you are asked to use the MIPS calling convention (as explained in this reading and in the lectures) when writing assembly code for this course, then *please ensure that you do so*. On an assignment or a quiz or an exam, if your code violates the MIPS calling convention, **even if it works**, then you can expect a majorly large deduction of points (up to and including issuing a zero grade) on that problem/question.

This concludes the learning material for MIPS assembly program for our course!